Lecture 8:

Data-Parallel Thinking

Parallel Computing

Today's theme

- Many of you are now likely accustomed to thinking about parallel programming in terms of "what workers do"

- Today I would like you to think about describing algorithms in terms of *operations on sequences of data*
  - map
  - filter
  - fold / reduce
  - scan / segmented scan
  - sort
  - groupBy
  - join
  - partition / flatten

- Main idea: high-performance parallel implementations of these operations exist. So programs written in terms of these primitives can often run efficiently on a parallel machine *

* if you can avoid being bandwidth bound

Motivation

- Why must an application expose large amounts of parallelism?

- Utilize large numbers of cores
  - High-core count machines
  - Many machines (e.g., cluster of machines in the cloud)
  - SIMD processing + multi-threaded cores require even more parallelism
  - GPU architectures require very large amounts of parallelism

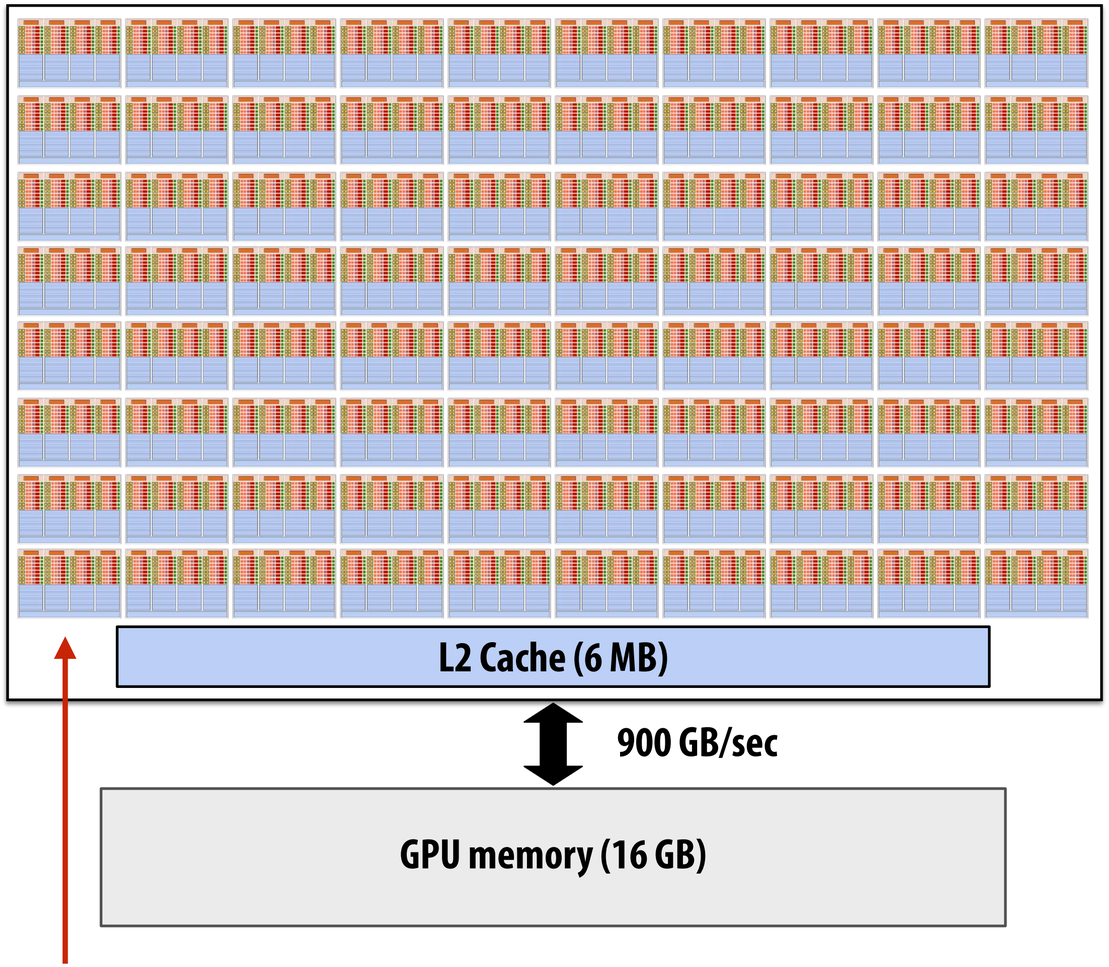
Recall: geometry of the V100 GPU

1.245 GHz clock

80 SM cores per chip

80 x 4 x 16 = 5,120 fp32 mul-add ALUs
= 12.7 TFLOPs *

Up to 80 x 64 = 5120 interleaved warps per chip (163,840 CUDA threads/chip)



This chip can concurrently execute up to 163,860 CUDA threads! (programs that do not expose significant amounts of parallelism, and don't have high arithmetic intensity, will not run efficiently on GPUs!)

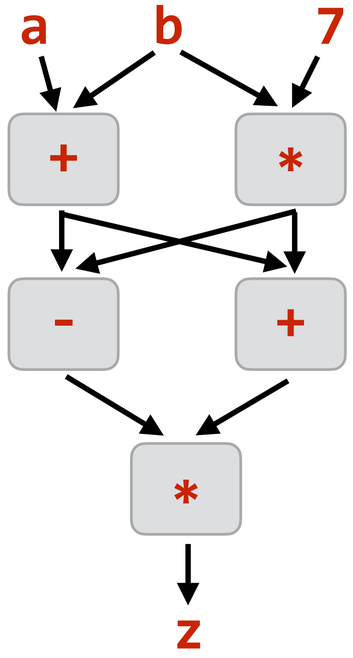
* mul-add counted as 2 flops:

Understanding dependencies is key

- Key part of parallel programming is understanding when dependencies exist between operation

- Lack of dependencies implies potential for parallel execution

```
x = a + b;
y = b * 7;
z = (x-y) * (x+y);
```



Data-parallel model

- Organize computation as operations on sequences of elements
  - e.g., perform same function on all elements of a sequence

- A well-known modern example: NumPy: C = A + B
  (A, B, and C are vectors of same length)

Key data type: sequences

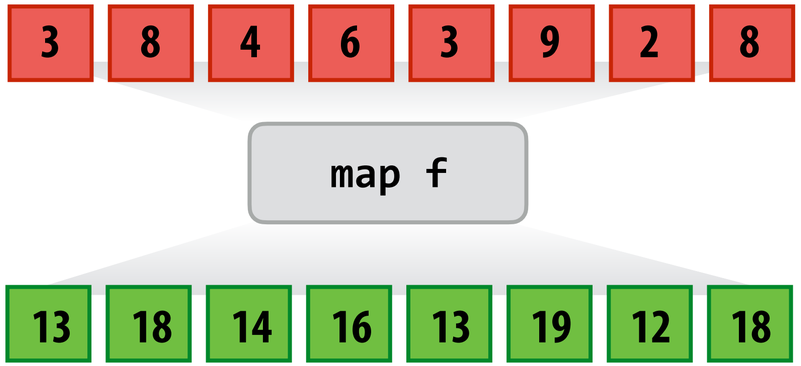
- Ordered collection of elements
- In a C++ like language: Sequence<T>
- Scala lists: List[T]
- Python Pandas Dataframes
- In a functional language (like Haskell): seq T

- Important: unlike arrays, programs can only access elements of a sequence through specific operations

Map

- Higher order function (function that takes a function as an argument)
- Applies side-effect free unary function `f :: a -> b` to all elements of input sequence, producing output sequence of the same length
- In a functional language (e.g., Haskell)
  - `map :: (a -> b) -> seq a -> seq b`
- In C++:

```
template<class InputIt, class OutputIt, class UnaryOperation>
OutputIt transform(InputIt first1, InputIt last1, OutputIt d_first,
                   UnaryOperation unary_op);
```



C++

```
int f(int x) { return x + 10; }

int a[] = {3, 8, 4, 6, 3, 9, 2, 8};
int b[8];
std::transform(a, a+8, b, f);
```

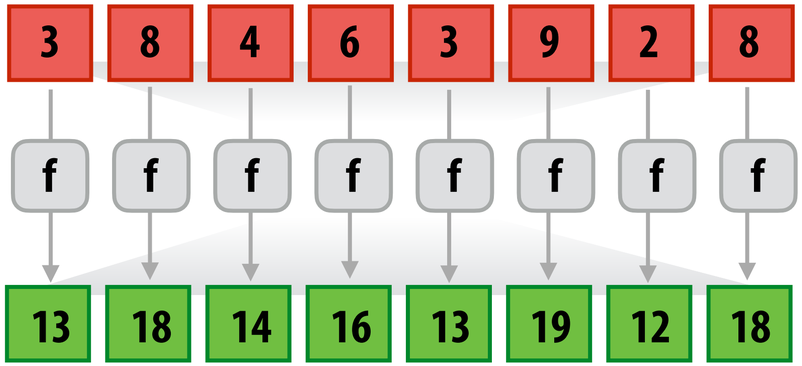
Output start iterator

Input end iterator

Input start iterator

Haskell

```
a = [3, 8, 4, 6, 3, 9, 2, 8]
f x = x + 10
b = map f a
```



Parallelizing map

- Since `f :: a -> b` is a function (side-effect free), then applying `f` to all elements of the sequence can be performed in *any order* without changing the output of the program

- The implementation of map has flexibility to reorder/parallelize processing of elements of sequence however it sees fit

```
map f s =
    partition sequence s into P smaller sequences
    for each subsequence s_i (in parallel)
        out_i = map f s_i
    out = concatenate out_i's
```

Fold (fold left)

- Apply binary operation f to each element and an accumulated value
  - Seeded by initial value of type b

```
f :: (b,a) -> b
fold :: b -> ((b,a) -> b) -> seq a -> b
```

Output

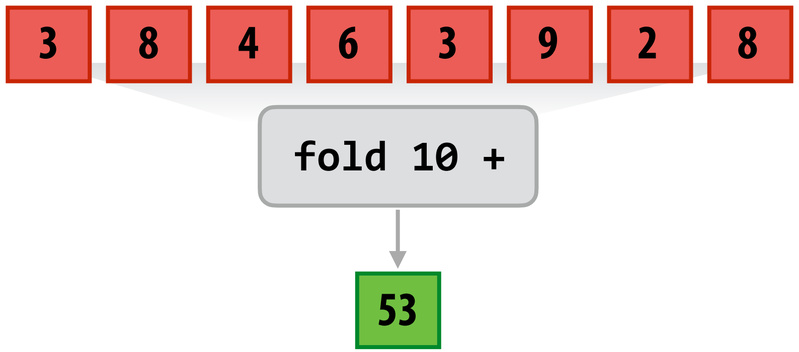
Initial element

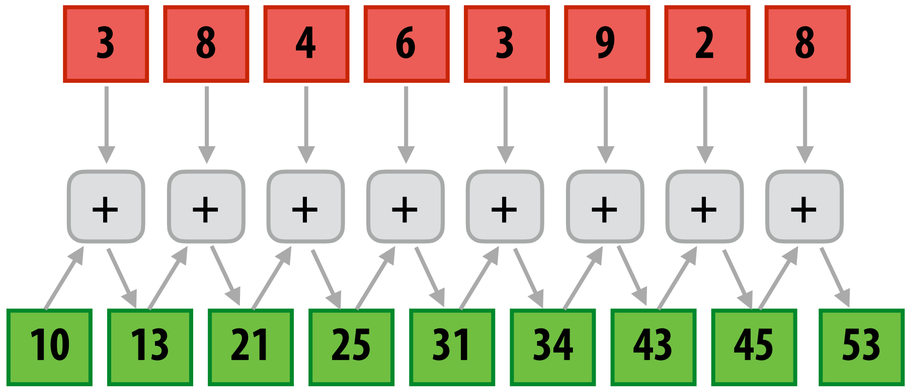
Function to fold

Input sequence

E.g., in Scala:

```
def foldLeft[A, B](init: B, f: (B, A) => B, l: List[A]): B
```

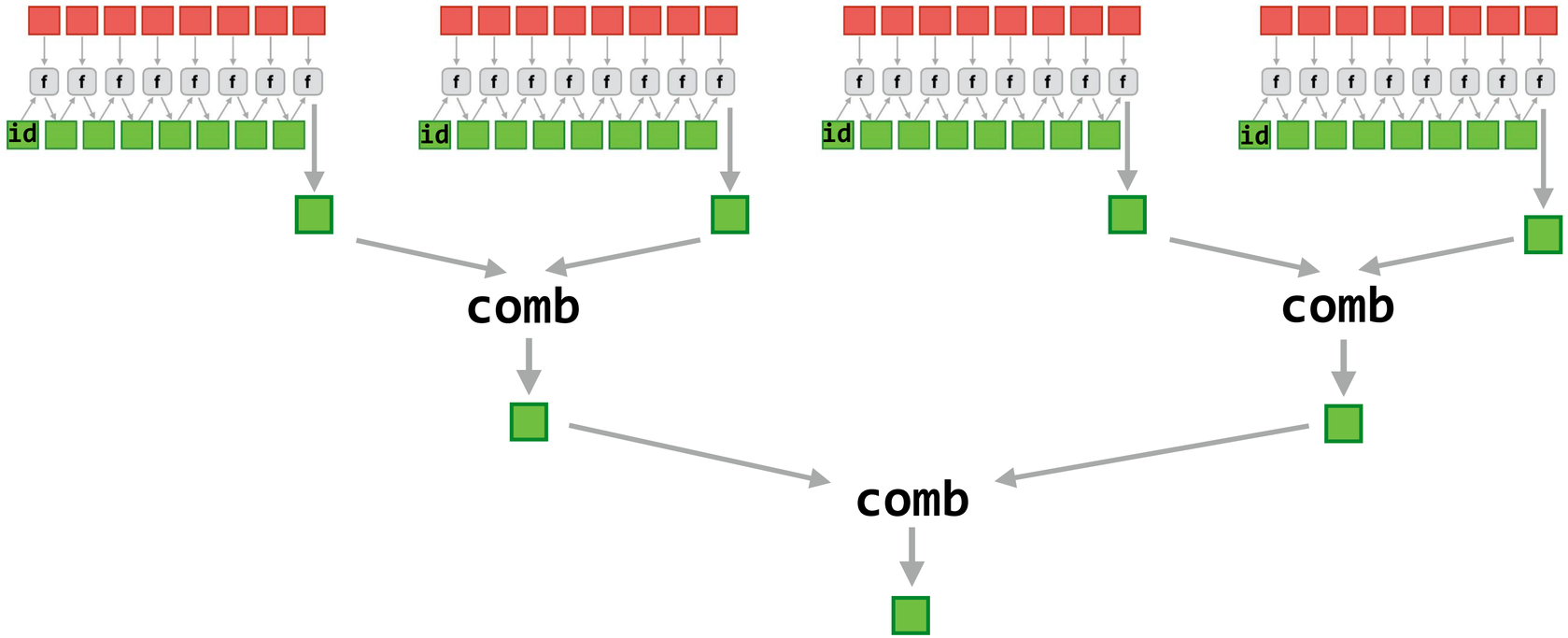




Parallel fold

- Apply f to each element and an accumulated value
  - In addition to binary function f, also need an additional binary "combiner" function *
  - Seeded by initial value of type b (must be identity for *f* and *comb*)

```
f :: (b,a) -> b
comb :: (b,b) -> b
fold_par :: b -> ((b,a) -> b) -> ((b,b)->b) ->seq a -> b
```

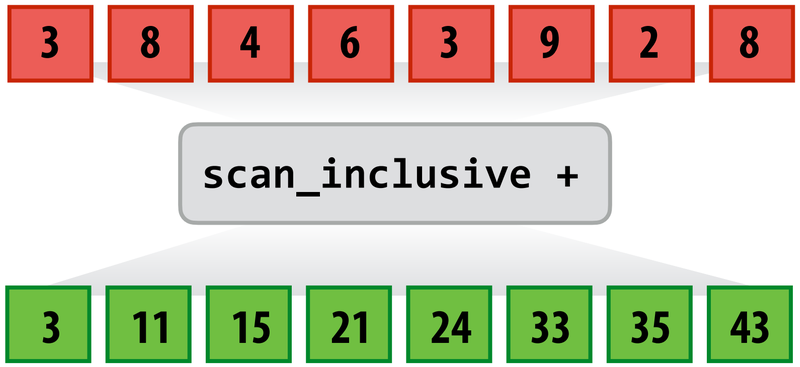


* No need for comb if `f::(b,b)->b` is an associative binary operator

Scan

```
f :: (a,a) -> a
```
(associative binary op)

```
scan :: a -> ((a,a) -> a) -> seq a -> seq a
```




```
float op(float a, float b) { … }
scan_inclusive(float* in, float* out, int N) {
 out[0] = in[0];
 for (i=1; i<N; i++)
   out[i] = op(out[i-1], in[i]);
}
```

Alternative form: "scan exclusive": out[i] is the scan result for all elements up to, but excluding, in[i].

Parallel Scan

Data-parallel scan

let $A = [a_0, a_1, a_2, a_3, \ldots, a_{n-1}]$

let $\oplus$ be an associative binary operator with identity element I

$scan_inclusive(\oplus, A) = [a_0, a_0 \oplus a_1, a_0 \oplus a_1 \oplus a_2, \ldots$

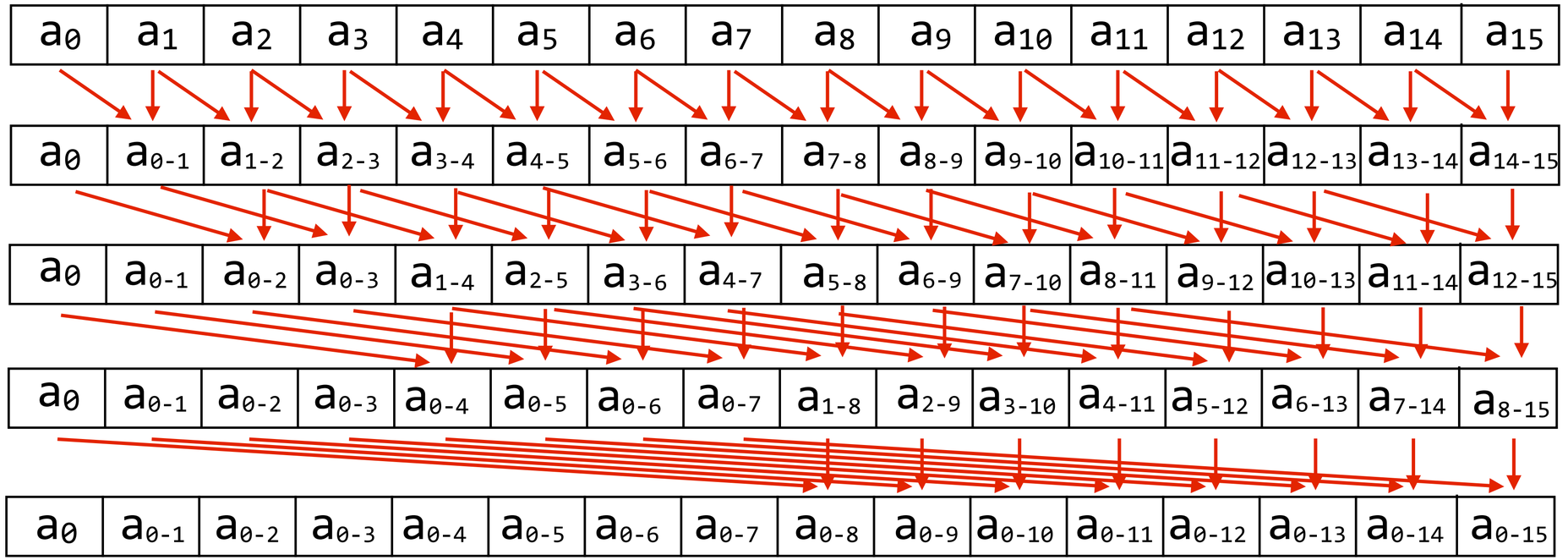
$scan_exclusive(\oplus, A) = [I, a_0, a_0 \oplus a_1, \ldots$

If operator is +, then `scan_inclusive(+,A)` is called "a prefix sum"

$prefix_sum(A) = [a_0, a_0+a_1, a_0+a_1+a_2, \ldots$

Data-parallel inclusive scan

(Subtract original vector to get exclusive scan result: not shown)



Total operations performed

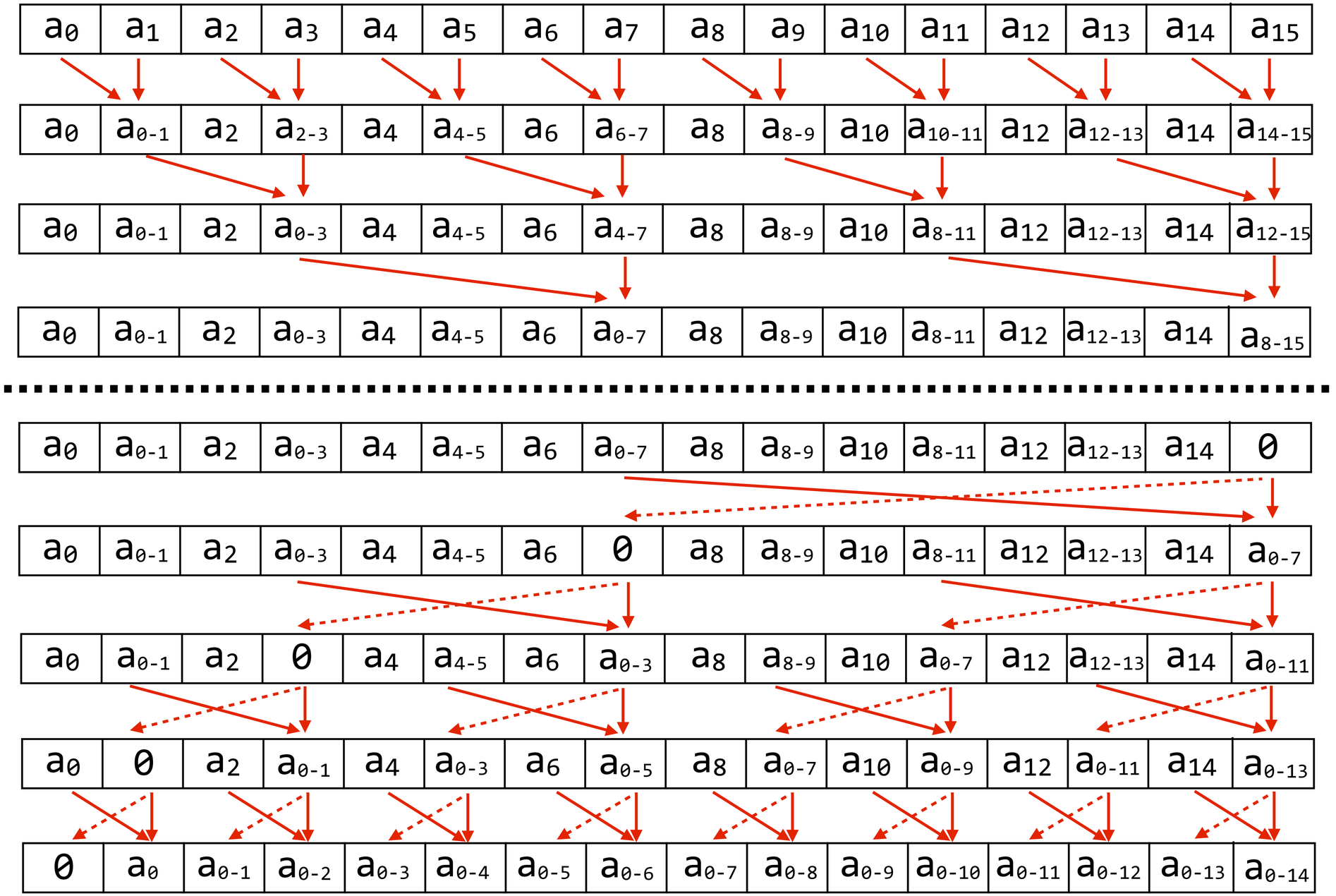
Work: O(N lg N)

Span: O(lg N)

Longest chain of sequential steps

Inefficient compared to sequential algorithm!

Work-efficient parallel exclusive scan (O(N) work)



Work efficient exclusive scan algorithm

(with $\oplus$ = "+")

Up-sweep:

```
for d=0 to (log2n - 1) do
   forall k=0 to n-1 by 2^(d+1) do
     a[k + 2^(d+1) - 1] = a[k + 2^d - 1] + a[k + 2^(d+1) - 1]
```

Down-sweep:

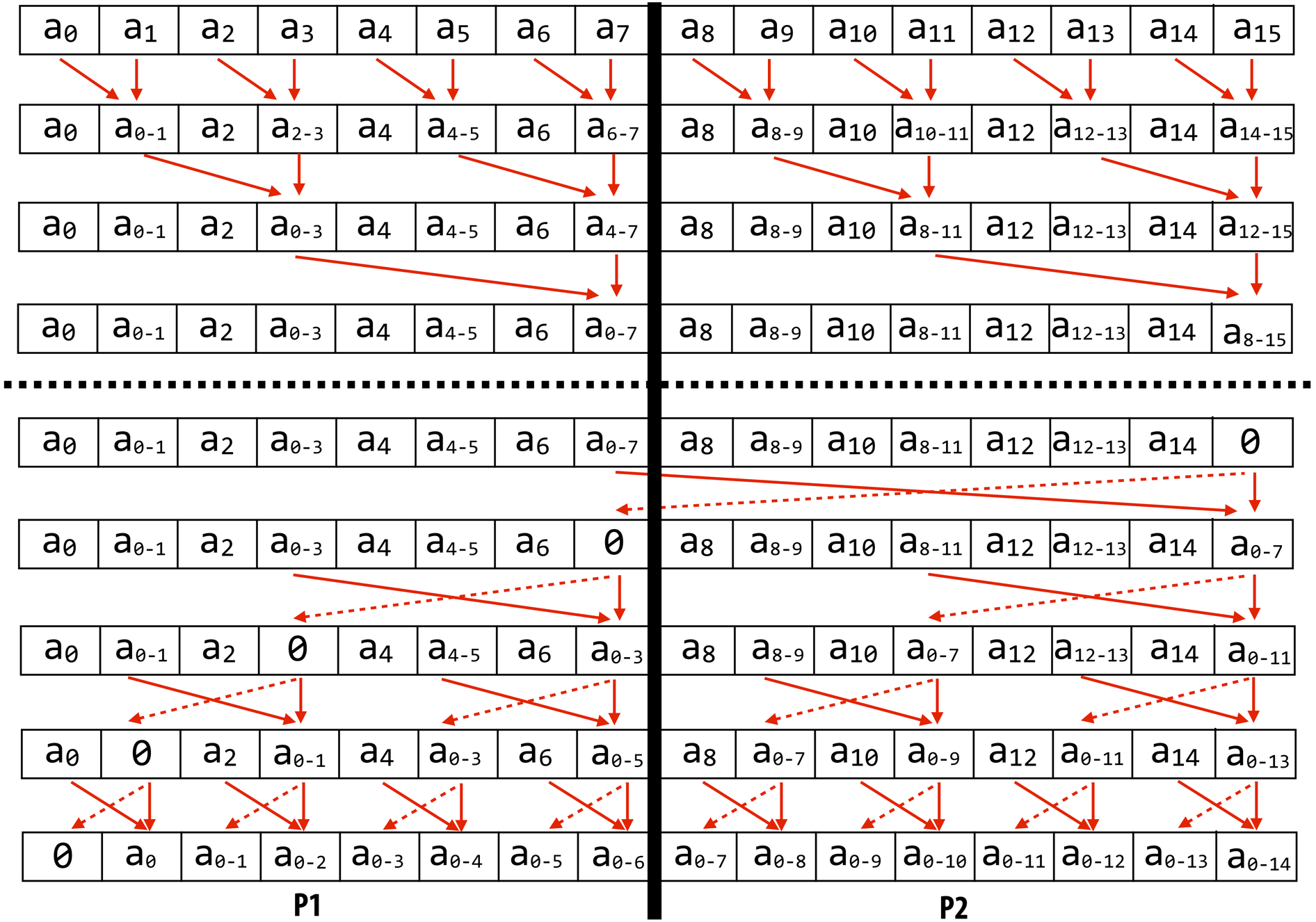
```
x[n-1] = 0
for d=(log2n - 1) down to 0 do
   forall k=0 to n-1 by 2^(d+1) do
     tmp = a[k + 2^d - 1]
     a[k + 2^d - 1] = a[k + 2^(d+1) - 1]
     a[k + 2^(d+1) - 1] = tmp + a[k + 2^(d+1) - 1]
```

Work: O(N) (but what is the constant?)

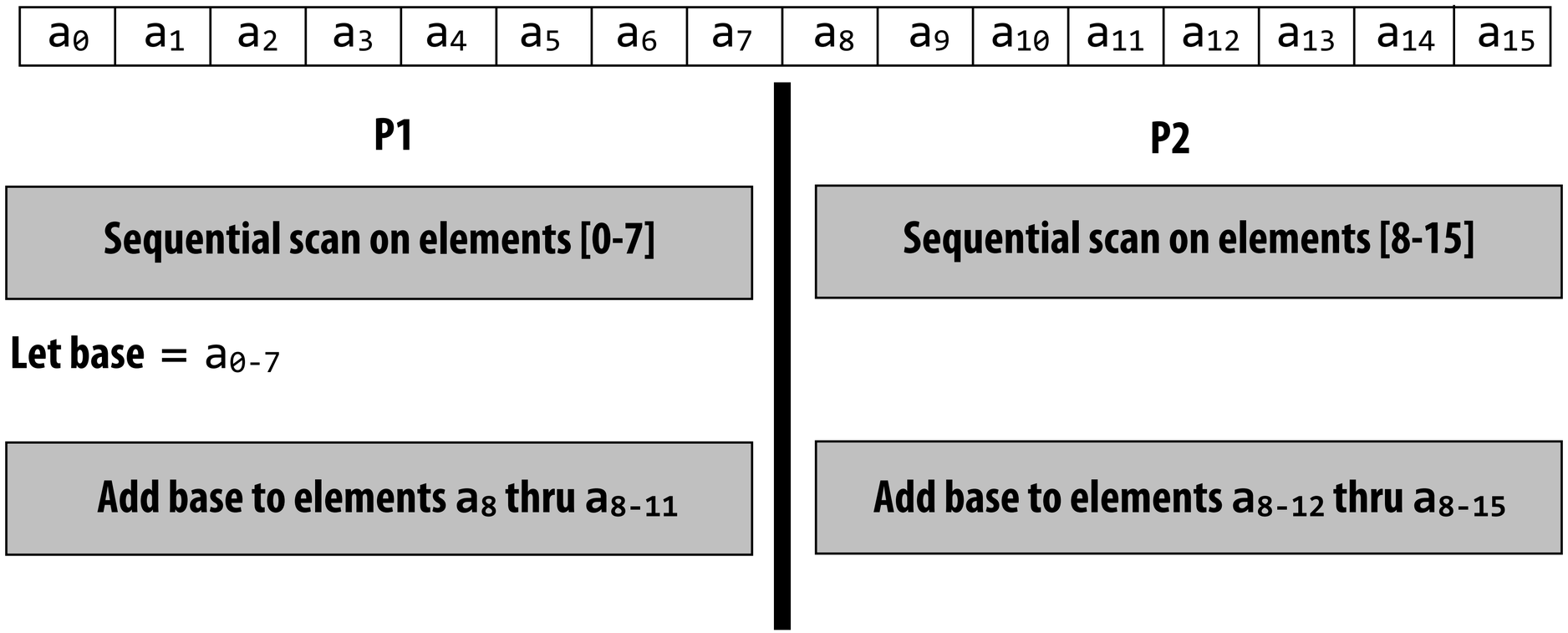
Span: O(lg N) (but what is the constant?)

Locality: ??

Now consider scan implementation on just two cores



Scan: two processor (shared memory) implementation



Work: O(N) **(but constant is now only 1.5)**

Data-access:

- **Very high spatial locality (contiguous memory access)**
- **P1's access to a_8 through a_{8-11} may be more costly on large core count system with non-uniform memory access costs, but on small-scale multi-core system the access cost is likely the same as from P2**

Exclusive scan: SIMD implementation (in CUDA)

Example: perform exclusive scan on 32-element array: SPMD program, assume 32-wide SIMD execution

When `scan_warp` is run by a group of 32 CUDA threads, each thread returns the exclusive scan result for element `idx`

(also: upon completion `ptr[ ]` stores inclusive scan result)

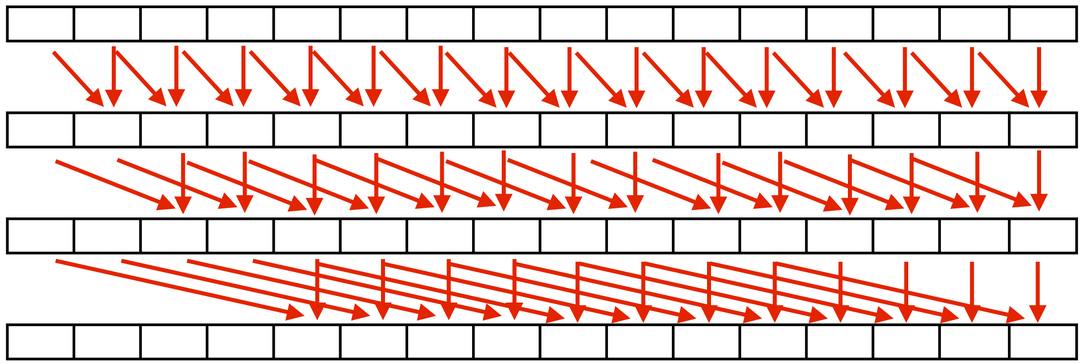
CUDA thread index of caller

```
__device__ int scan_warp(int *ptr, const unsigned int idx)
{
   const unsigned int lane = idx % 32; // index of thread in warp (0..31)

   if (lane >= 1)  ptr[idx] = ptr[idx - 1]  +  ptr[idx];
   if (lane >= 2)  ptr[idx] = ptr[idx - 2]  +  ptr[idx];
   if (lane >= 4)  ptr[idx] = ptr[idx - 4]  +  ptr[idx];
   if (lane >= 8)  ptr[idx] = ptr[idx - 8]  +  ptr[idx];
   if (lane >= 16) ptr[idx] = ptr[idx - 16] +  ptr[idx);

   return (lane > 0) ? ptr[idx-1] : 0;
}
```

Work: ??

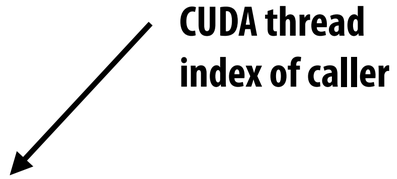


• • •

Exclusive scan: SIMD implementation (in CUDA)

Example: exclusive scan 32-element array

32-wide GPU execution (SPMD program)




```
__device__ int scan_warp(int *ptr, const unsigned int idx)
{
   const unsigned int lane = idx % 32; // index of thread in warp (0..31)

   if (lane >= 1)  ptr[idx] = ptr[idx - 1]  +  ptr[idx];
   if (lane >= 2)  ptr[idx] = ptr[idx - 2]  +  ptr[idx];
   if (lane >= 4)  ptr[idx] = ptr[idx - 4]  +  ptr[idx];
   if (lane >= 8)  ptr[idx] = ptr[idx - 8]  +  ptr[idx];
   if (lane >= 16) ptr[idx] = ptr[idx - 16] +  ptr[idx];

   return (lane > 0) ? ptr[idx-1] : 0;
}
```

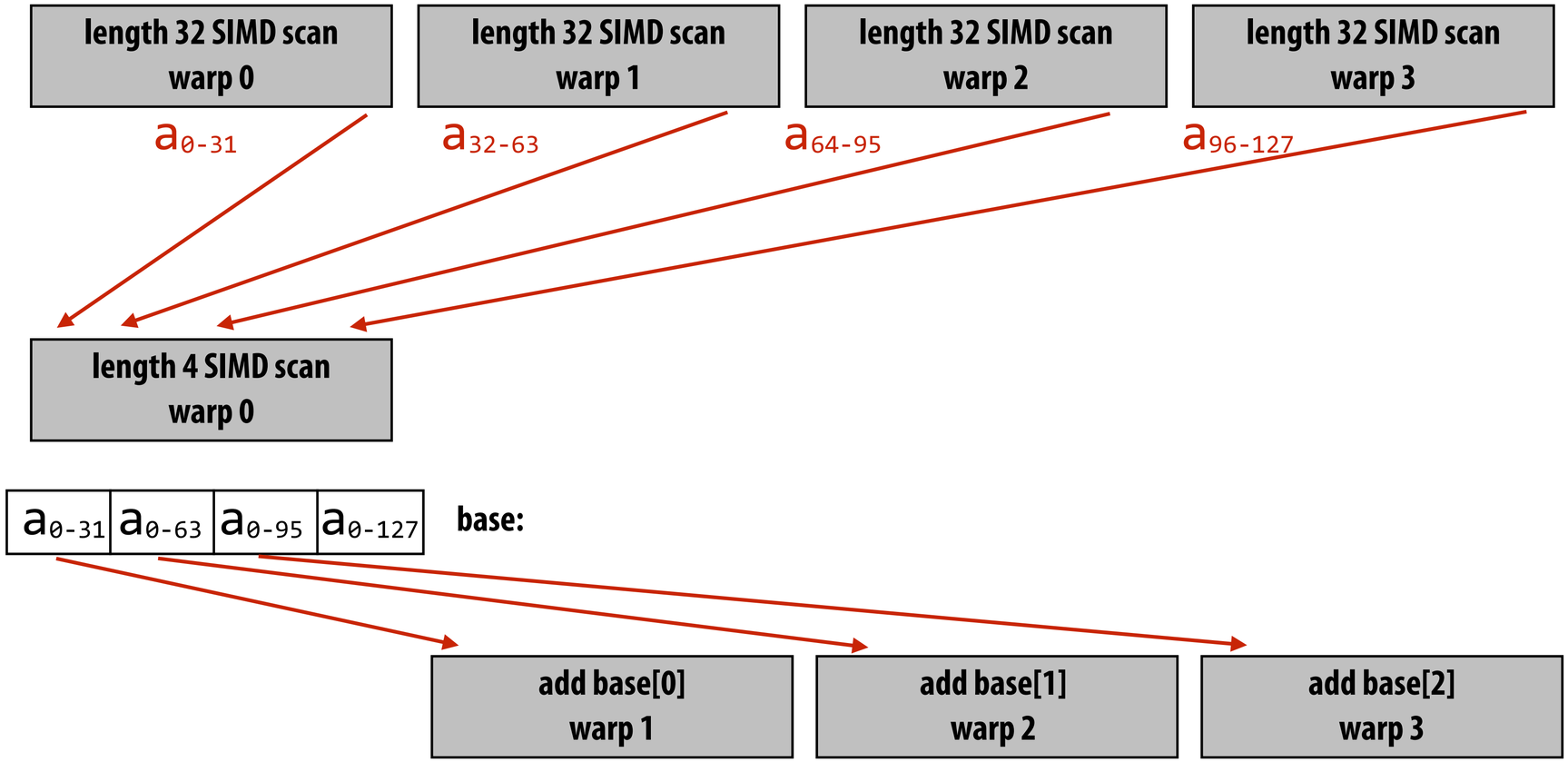
Work: N lg(N)

Work-efficient formulation of scan is not beneficial in this context because it results in low SIMD utilization.

Work efficient algorithm would require more than 2x the number of instructions as the implementation above!

Building scan on larger array

Example: 128-element scan using four-warp thread block



Multi-threaded, SIMD CUDA implementation

Example: cooperating threads in a CUDA thread block perform scan

We provide similar code in assignment 3.

Code assumes length of array given by `ptr` is same as number of threads per block.

CUDA thread index of caller

```
__device__ void scan_block(int* ptr, const unsigned int idx)
{
    const unsigned int lane = idx % 32;          // index of thread in warp (0..31)
    const unsigned int warp_id = idx >> 5;       // warp index in block

    int val = scan_warp(ptr, idx);               // Step 1. per-warp partial scan
                                                 // (Performed by all threads in block,
                                                 // with threads in same warp communicating
                                                 // through shared memory buffer 'ptr')

    if (lane == 31) ptr[warp_id] = ptr[idx];     // Step 2. thread 31 in each warp copies
    __syncthreads();                             // partial-scan result into per-block
                                                 // shared mem

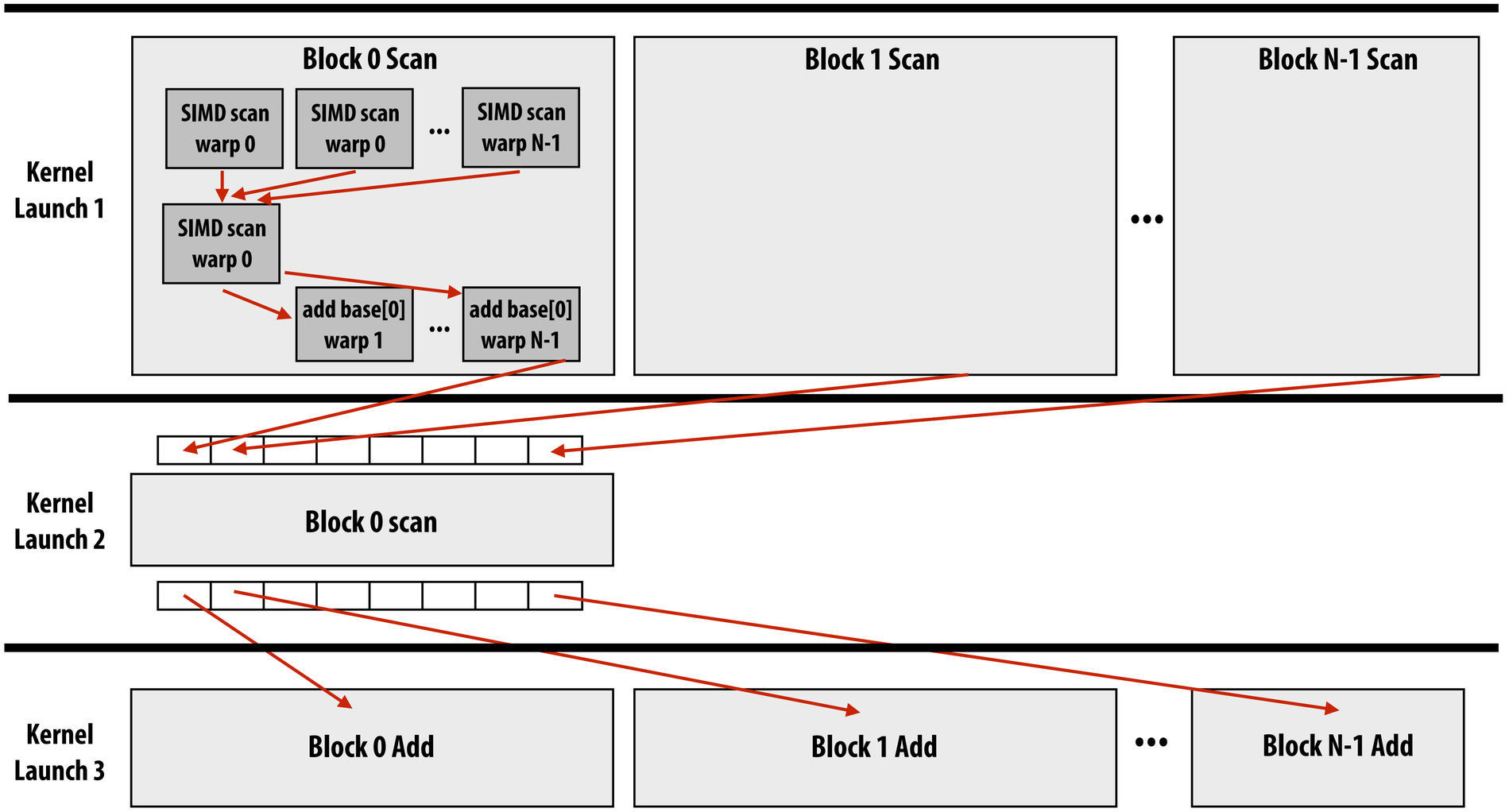
    if (warp_id == 0) scan_warp(ptr, idx);       // Step 3. scan to accumulate bases
    __syncthreads();                             // (only performed by warp 0)

    if (warp_id > 0)                             // Step 4. apply bases to all elements
        val = val + ptr[warp_id-1];              // (performed by all threads in block)
    __syncthreads();

    ptr[idx] = val;
}
```

Building a larger scan

Example: one million element scan (1024 elements per block)



Exceeding 1 million elements requires partitioning phase two into multiple blocks

Scan implementation

- **Parallelism**
  - Scan algorithm features O(N) parallel work
  - But efficient implementations only leverage as much parallelism as required to make good utilization of the machine
    - Goal is to reduce work and reduce communication/synchronization
- **Locality**
  - Multi-level implementation to match memory hierarchy
    (CUDA example: per-block implementation carried out in local memory)
- **Heterogeneity in algorithm: different strategy for performing scan at different levels of the machine**
  - CUDA example: different algorithm for intra-warp scan than inter-thread scan
  - Low-core count CPU example: based largely on sequential scan

Parallel Segmented Scan

Segmented scan

- Common problem: operating on a *sequence of sequences*
- Examples:
  - For each vertex v in a graph:
    - For each edge e connected to v:
  - For each particle p in a simulation
    - For each particle within distance D of p
  - For each document d in a collection
    - For each word in d
- There are two levels of parallelism in the problem that a programmer might want to exploit
- But it is irregular: the size of edge lists, particle neighbor lists, words per document, etc, may be very different from vertex to vertex (or particle to particle)

Segmented scan

- Generalization of scan
- Simultaneously perform scans on contiguous partitions of input sequence

```
let A  = [[1,2],[6],[1,2,3,4]]
let ⊕ = +
segmented_scan_exclusive(⊕,A) = [[0,1], [0], [0,1,3,6]]
```

Assume a simple "start-flag" representation of nested sequences:

Consider nested sequence `A = [[1,2,3],[4,5,6,7,8]]`

```
flag: 1 0 0 1 0 0 0 0
data: 1 2 3 4 5 6 7 8
```

Work-efficient segmented scan

(with $\oplus$ = "+")

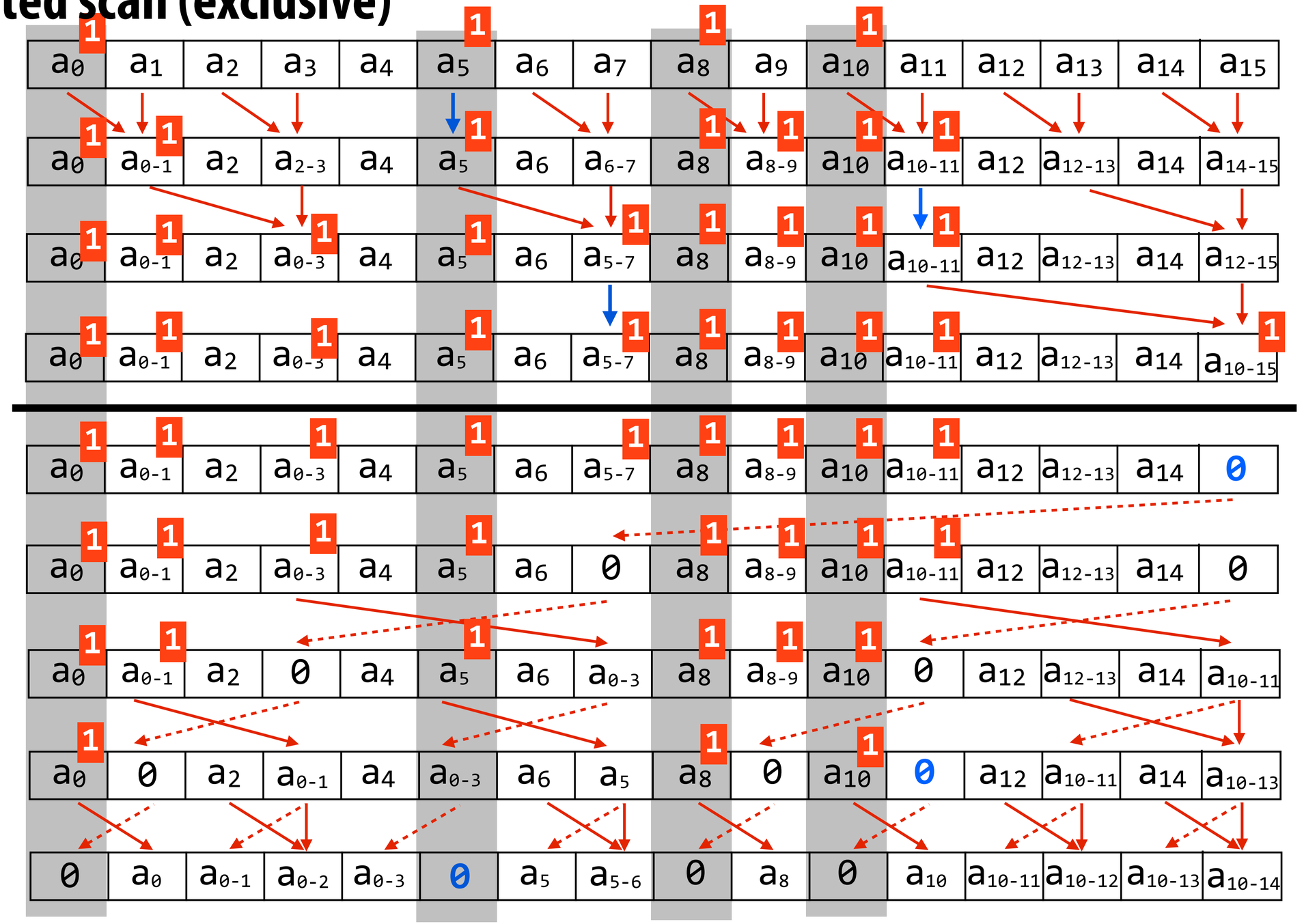
Up-sweep:

```
for d=0 to (log2n - 1) do:
   forall k=0 to n-1 by 2^(d+1) do:
     if flag[k + 2^(d+1) - 1] == 0:
        data[k + 2^(d+1) - 1] = data[k + 2^d - 1] + data[k + 2^(d+1) - 1]
     flag[k + 2^(d+1) - 1] = flag[k + 2^d - 1] || flag[k + 2^(d+1) - 1]
```

Down-sweep:

```
data[n-1] = 0
for d=(log2n - 1) down to 0 do:
   forall k=0 to n-1 by 2^(d+1) do:
     tmp = data[k + 2^d - 1]
     data[k + 2^d - 1] = data[k + 2^(d+1) - 1]
     if flag_original[k + 2^d] == 1:       # must maintain copy of original flags
        data[k + 2^(d+1) - 1] = 0           # start of segment
     else if flag[k + 2^d - 1] == 1:
        data[k + 2^(d+1) - 1] = tmp
     else:
         data[k + 2^(d+1) - 1] = tmp + data[k + 2^(d+1) - 1]
     flag[k + 2^d - 1] = 0
```

Segmented scan (exclusive)



Sparse matrix multiplication example

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 3 & 0 & 1 & \cdots & 0 \\ 0 & 2 & 0 & \cdots & 0 \\ 0 & 0 & 4 & \cdots & 0 \\ & & \vdots & & \\ 0 & 2 & 6 & \cdots & 8 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{bmatrix}$$

- **Most values in matrix are zero**
  - Note: easy parallelization by parallelizing the different per-row dot products
  - But different amounts of work per row (complicates wide SIMD execution)
- **Example sparse storage format: compressed sparse row**

values = [[3,1], [2], [4], ..., [2,6,8]]

cols = [[0,2], [1], [2], ...,]

row_starts = [0, 2, 3, 4, ...]

Sparse matrix multiplication with scan

x = [x0,x1,x2,x3]

values = [[3,1], [2], [4], [2,6,8]]

cols = [[0,2], [1], [2], [1,2,3]]

row_starts = [0, 2, 3, 4]

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 3 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 2 & 6 & 8 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

1. Map over all non-zero values: products[i] = values[i] * x[cols[i]]

   gather(x, cols)

   products = [$3x_0$, x_2, $2x_1$, $4x_2$, $2x_1$, $6x_2$, $8x_3$]
2. Create flags vector from row_starts: flags = [1,0,1,1,1,0,0]
3. Perform <u>inclusive</u> segmented-scan on (products, flags) using addition operator

   [$3x_0$, $3x_0+x_2$, $2x_1$, $4x_2$, $2x_1$, $2x_1+6x_2$, $2x_1+6x_2+8x_2$]
4. Take last element in each segment:

   y = [$3x_0+x_2$, $2x_1$, $4x_2$, $2x_1+6x_2+8x_2$]

Scan/segmented scan summary

- Scan
  - Theory: parallelism in problem is linear in number of elements
  - Practice: exploit locality, use only as much parallelism as necessary to fill the machine's execution resources
    - Great example of applying different strategies at different levels of the machine
- Segmented scan
  - Express computation and operate on irregular data structures (e.g., list of lists) in a regular, data parallel way

Gather/scatter: key data-parallel operations

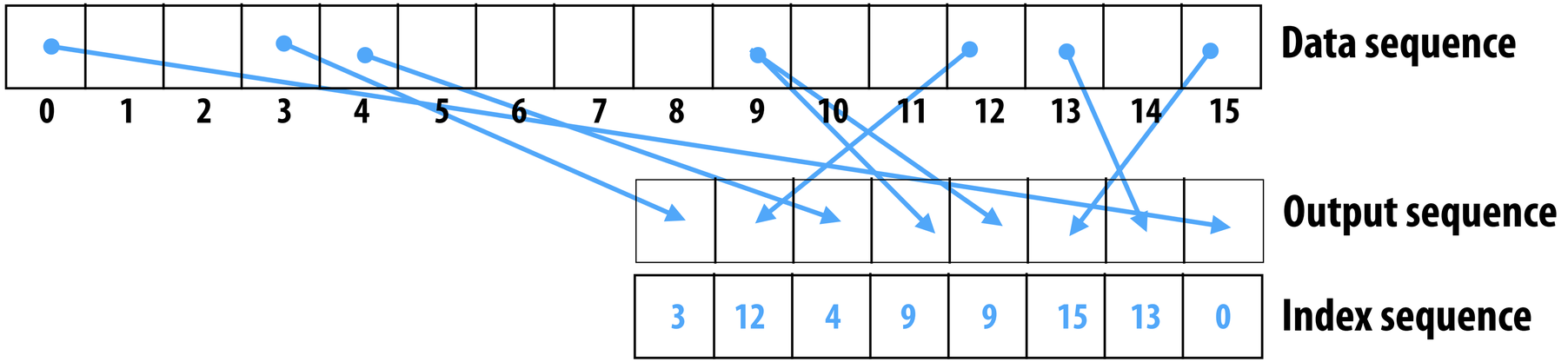
- **gather(index, input, output)**
  - `output[i] = input[index[i]]`

- **scatter(index, input, output)**
  - `output[index[i]] = input[i]`

Gather/scatter: key data-parallel operations

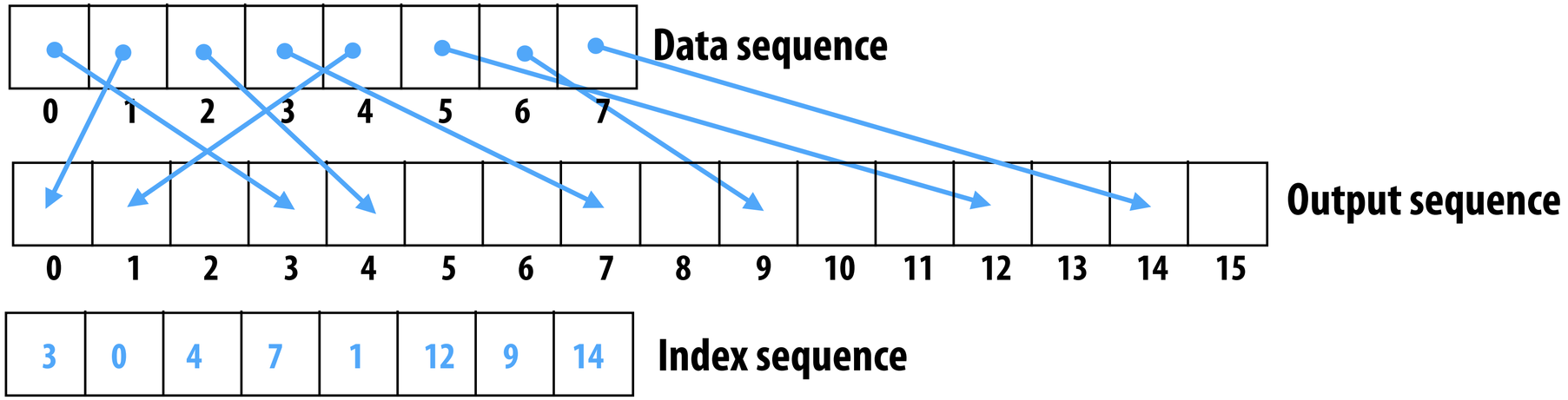
```
output_seq = gather(index_seq, data_seq)
```

"Gather data from data_seq according to indices in index_seq"



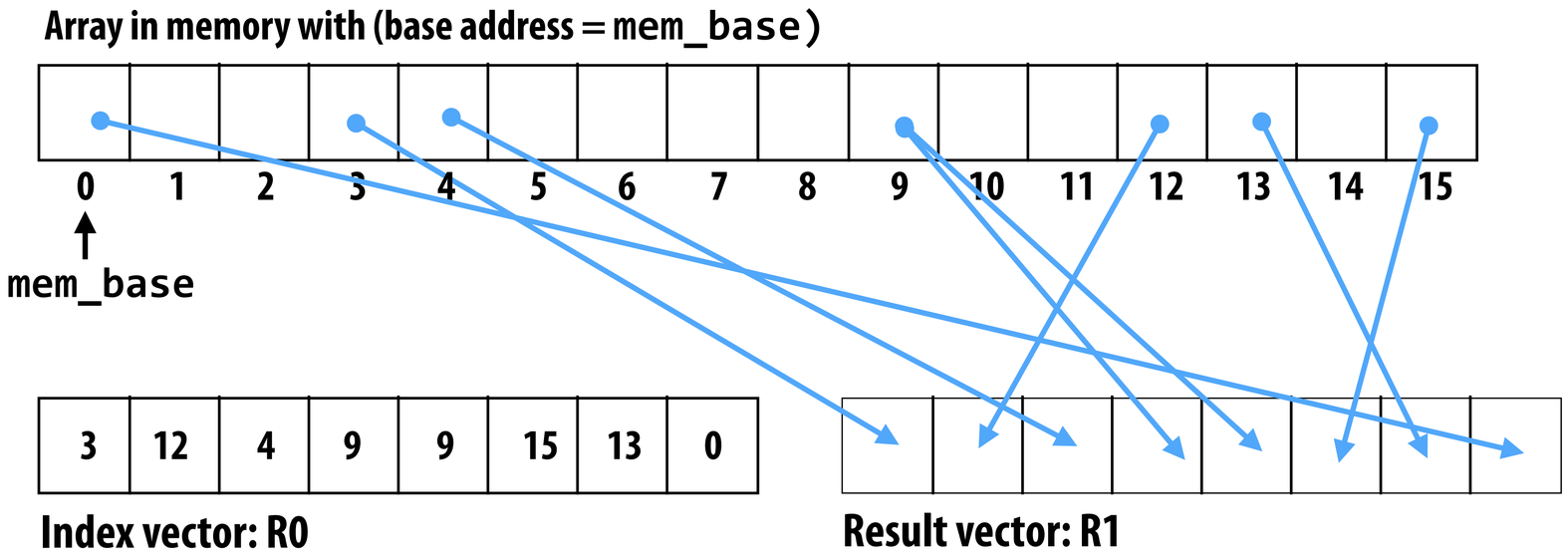

```
output_seq = scatter(index_seq, data_seq)
```

"Scatter data from data_seq according to indices in index_seq"



Gather machine instruction

`gather(R1, R0, mem_base);` *"Gather from buffer `mem_base` into R1 according to indices specified by R0."*



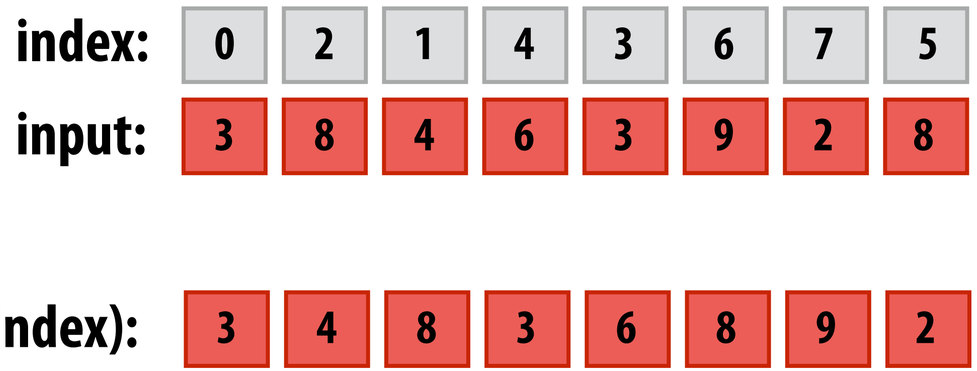
Gather supported with AVX2 in 2013
But AVX2 does not support SIMD scatter (must implement as scalar loop)
Scatter instruction exists in AVX512

Hardware supported gather/scatter exists on GPUs.
(still an expensive operation compared to load/store of contiguous vector)

Turning a scatter into sort/gather

Special case: assume elements of index are unique and all elements referenced in index (scatter is a permutation)

```
scatter(index, input, output) {
  output = sort input sequence by values in index sequence
}
```



Implementing scatterOp with atomic sort/map/segmented-scan

Now, assume elements in index are not unique, so synchronization is required for atomicity!

```
for all elements in sequence
    output[index[i]] = atomicOp(output[index[i]], input[i])
```

Example:
atomicAdd(output[index[i], input[I]])

`e.g,: index = [1, 1, 0, 2, 0, 0]`

Step 1: Sort input sequence according to values in index sequence:

Sorted index:

`[0, 0, 0, 1, 1, 2]`

Input sorted by index:

`[input[2], input[4], input[5], input[0], input[1], input[3]]`

Step 2: Compute starts of each range of values with the same index number

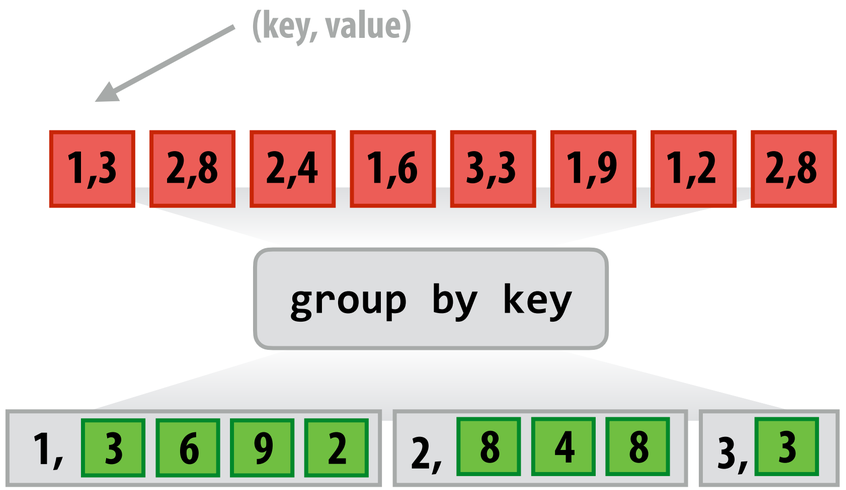
`starts: [1, 0, 0, 1, 0, 1]`

Step 3: Segmented scan (using 'op') on each range

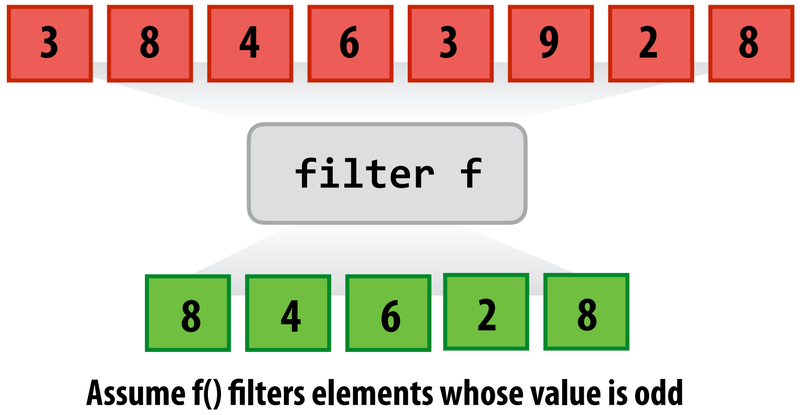
`[op(op(input[2], input[4]), input[5]), op(input[0], input[1]), input[3])`

More sequence operations

- Group by key
  - Seq (key, T) —> Seq (key, Seq T)
  - Creates a sequence of sequences containing elements with the same key



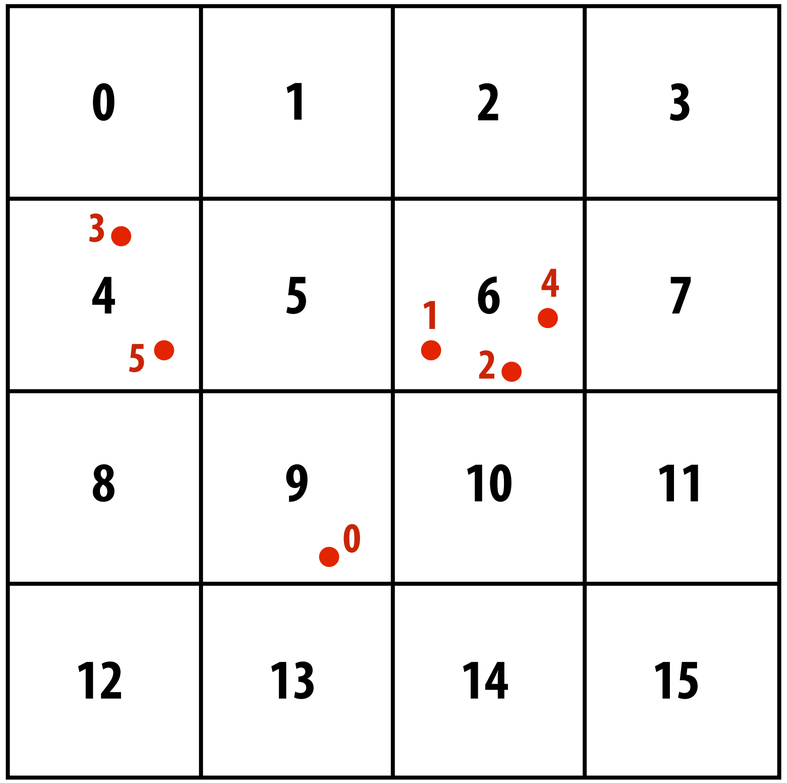
- Filter
  - Remove elements from sequence that do not match predicate



- Sort

Example: create grid of particles data structure on large parallel machine (e.g., a GPU)

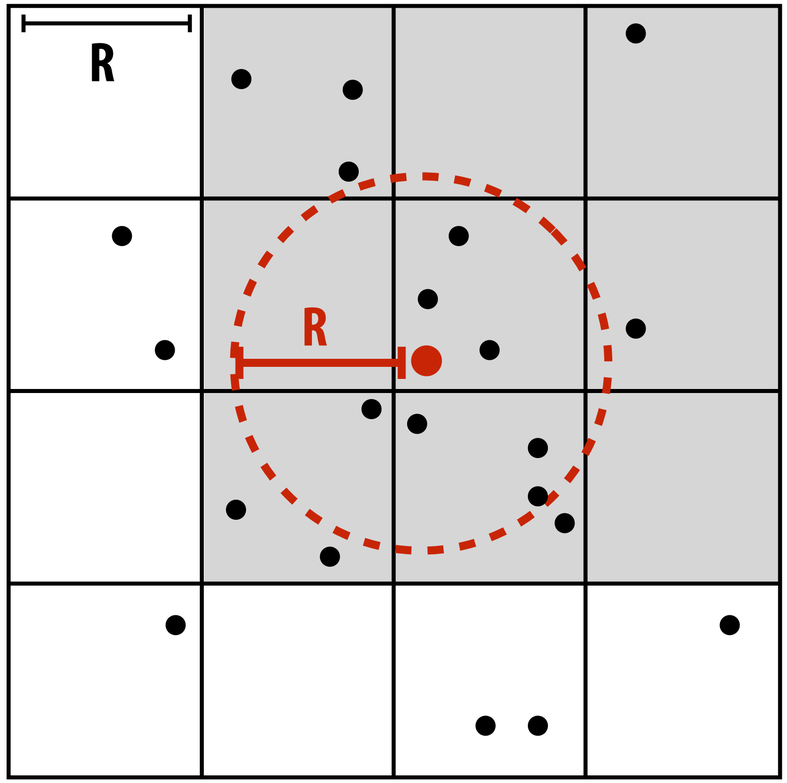
- Problem: place 1M point particles in a 16-cell uniform grid based on 2D position
  - Parallel data structure manipulation problem: build a 2D array of lists
- Recall: Up to 2048 CUDA threads per SM core on a V100 GPU (80 SM cores)



Cell id	Count	Particle id
0	0	
1	0	
2	0	
3	0	
4	2	3, 5
5	0	
6	3	1, 2, 4
7	0	
8	0	
9	1	0
10	0	
11	0	
12	0	
13	0	
14	0	
15	0	

Common use of this structure: N-body problems

- A common operation is to compute interactions with neighboring particles
- Example: given a particle, find all particles within radius R
  - Organize particles by placing them in grid with cells of size R
  - Only need to inspect particles in surrounding grid cells



Solution 1: parallelize over particles

- One answer: assign one particle to each CUDA thread. Each thread computes cell containing particle, then atomically updates per cell list.
  - Massive contention: thousands of threads contending for access to update single shared data structure

```
list cell_list[16];    // 2D array of lists
lock cell_list_lock;

for each particle p           // in parallel
   c = compute cell containing p
   lock(cell_list_lock)
   append p to cell_list[c]
   unlock(cell_list_lock)
```

Solution 2: use finer-granularity locks

- Alleviate contention for single global lock by using per-cell locks
  - Assuming uniform distribution of particles in 2D space... ~16x less contention than previous solution

```
list cell_list[16];    // 2D array of lists
lock cell_list_lock[16];

for each particle p           // in parallel
   c = compute cell containing p
   lock(cell_list_lock[c])
   append p to cell_list[c]
   unlock(cell_list_lock[c])
```

Solution 3: parallelize over cells

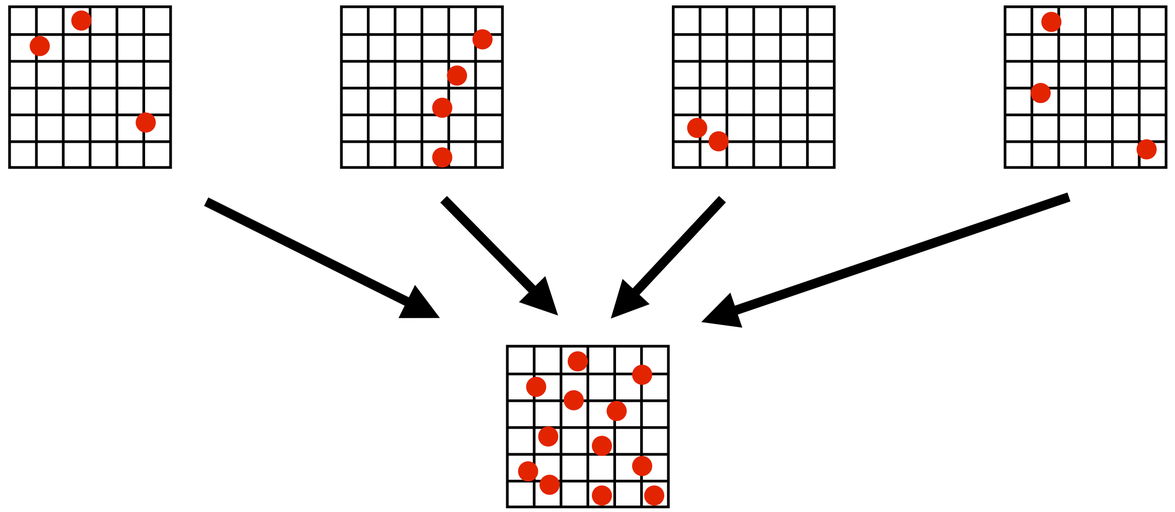
- Decompose work by cells: for each cell, independently compute what particles are within it (eliminates contention because no synchronization is required)
  - Insufficient parallelism: only 16 parallel tasks, but need thousands of independent tasks to efficiently utilize GPU)
  - Work inefficient: performs 16 times more particle-in-cell computations than sequential algorithm

```
list cell_lists[16];       // 2D array of lists

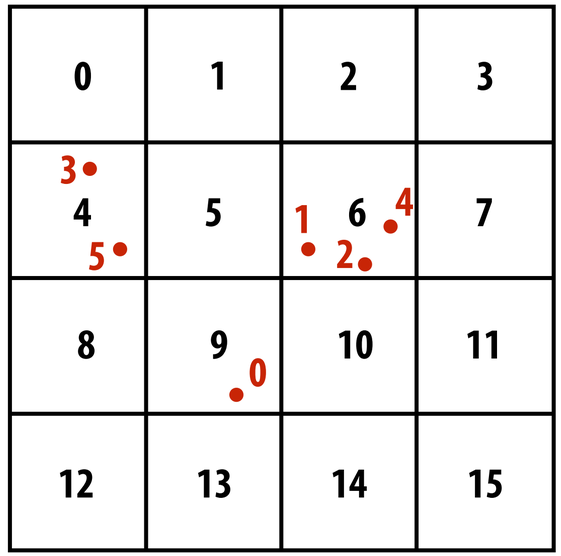
for each cell c            // in parallel
   for each particle p     // sequentially
       if (p is within c)
          append p to cell_lists[c]
```

Solution 4: compute partial results + merge

- Yet another answer: generate N "partial" grids in parallel, then combine
  - Example: create N thread blocks (at least as many thread blocks as SM cores)
  - All threads in thread block update same grid
    - Enables faster synchronization: contention reduced by factor of N and cost of synchronization is lower because it is performed on block-local variables (in CUDA shared memory)
  - Requires extra work: merging the N grids at the end of the computation
  - Requires extra memory footprint: stores N grids of lists, rather than 1



Solution 5: data-parallel approach



Step 1: map

compute cell containing each particle (parallel over input particles)

particle_index: 0 1 2 3 4 5

grid_cell:	9	6	6	4	6	4

Step 2: sort results by cell (notice that the particle index array is permuted based on sort)

particle_index: 3 5 1 2 4 0

grid_cell:	4	4	6	6	6	9

Step 3: find start/end of each cell (parallel over particle_index elements)

```
this_cell = grid_cell[thread_index];
if (thread_index == 0)                  // special case for first cell
    cell_starts[this_cell] = 0;
else if (this_cell != grid_cell[thread_index-1]) {
    cell_starts[this_cell] = thread_index;
    cell_ends[grid_cell[thread_index-1]] = thread_index;
}
if (thread_index == numParticles-1) // special case for last cell
    cell_ends[this_cell] = thread_index+1;
```

This code is run for each element of the particle_index array.
(each invocation has a unique valid of 'index')

This solution maintains a large amount of parallelism and removes the need for fine-grained synchronization…
at the cost of a sort and extra passes over the data (extra BW)!

cell_starts					0		2			5		…
cell_ends (not inclusive)					2		5			6		…
	0	1	2	3	4	5	6	7	8	9	10	

Another example: parallel histogram

- Consider computing a histogram for a sequence of values

```
int f(float value);              // maps array values to histogram bin id's

float input[N];
int histogram_bins[NUM_BINS];  // assume bins are initialized to 0

for (int i=0; i<N; i++) {
   histogram_bins[f(input[i])]++;
}
```

- Challenge: create a massively parallel implementation of histogram given only map() and sort() on sequences

Data-parallel histogram construction (part 1)

```
void compute_bin(float* input, int* bin_ids) {
   bin_ids[thread_index] = f(input[thread_index]);
}

void find_starts(int* bin_ids, int* bin_starts) {
   if (thread_index == 0 || bin_ids[thread_index] != bin_ids[thread_index-1])
      bin_starts[bin_ids[thread_index]] = thread_index;
}

float input[N];
int   histogram_bins[NUM_BINS];

// temporary buffers
int   bin_ids[N];              // bin_ids[i] = id of bin that element i goes in
int   sorted_bin_idx[N];
int   bin_starts[NUM_BINS];  // initialized to -1

// map f onto input sequence to get bin ids of all elements
launch<<<N>>>compute_bin(input, bin_ids);

// find starting point of each bin in sorted list
sort(N, bin_ids, sorted_bin_ids);
launch<<<N>>>find_starts(sorted_bin_ids, bin_starts);

// compute bin sizes (see definition of bin_sizes() on next slide)
launch<<<NUM_BINS>>>bin_sizes(bin_starts, histogram_bins, N, NUM_BINS);
```

Assume variable thread_index is the "thread index" associated with the invocation of the kernel function

Data-parallel histogram construction (part 2)

```
// launched with one thread per output bin
void bin_sizes(int* bin_starts, int* histogram_bins, int num_items, int num_bins) {

   if (bin_starts[thread_index] == -1) {
       histogram_bins[thread_index] = 0;    // no items in this bin
   } else {

       // find start of next bin in order to determined size of current bin

       // Tricky edge case: if the next bin is empty, then must search
       // forward to find the next non-empty bin
       int next_idx = thread_index+1;
       while(next_idx < num_bins && bin_starts[next_idx] == -1)
          next_idx++;

       if (next_idx < num_bins)
          histogram_bins[thread_index] = bin_starts[next_idx] - bin_starts[thread_index];
       else
          histogram_bins[thread_index] = num_items - bin_starts[thread_index];
      }
   }
}
```

Assume variable thread_index is the "thread index" associated with the invocation of the kernel function

Summary

- Data parallel thinking:
  - Implementing algorithms in terms of simple (often widely parallelizable, efficiently implemented) operations on large data collections

- Turn irregular parallelism into regular parallelism

- Turn fine-grained synchronization into coarse synchronization

- But most solutions require multiple passes over data — bandwidth hungry!

Summary

- Data parallel primitives are basis for many parallel/distributed systems today

- CUDA's Thrust Library
- Apache Spark / Hadoop
- Pandas Dataframe operations





Transformations	*map*(*f* : T ⇒ U)	:	RDD[T] ⇒ RDD[U]
	filter(*f* : T ⇒ Bool)	:	RDD[T] ⇒ RDD[T]
	flatMap(*f* : T ⇒ Seq[U])	:	RDD[T] ⇒ RDD[U]
	sample(*fraction* : Float)	:	RDD[T] ⇒ RDD[T] (Deterministic sampling)
	groupByKey()	:	RDD[(K, V)] ⇒ RDD[(K, Seq[V])]
	reduceByKey(*f* : (V, V) ⇒ V)	:	RDD[(K, V)] ⇒ RDD[(K, V)]
	union()	:	(RDD[T], RDD[T]) ⇒ RDD[T]
	join()	:	(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (V, W))]
	cogroup()	:	(RDD[(K, V)], RDD[(K, W)]) ⇒ RDD[(K, (Seq[V], Seq[W]))]
	crossProduct()	:	(RDD[T], RDD[U]) ⇒ RDD[(T, U)]
	mapValues(*f* : V ⇒ W)	:	RDD[(K, V)] ⇒ RDD[(K, W)] (Preserves partitioning)
	sort(*c* : Comparator[K])	:	RDD[(K, V)] ⇒ RDD[(K, V)]
	partitionBy(*p* : Partitioner[K])	:	RDD[(K, V)] ⇒ RDD[(K, V)]

- Iterators
  - Fancy Iterators
  - Iterator Tags
    - Iterator Tag Classes
- Algorithms
  - Searching
    - Binary Search
      - Vectorized Searches
  - Copying
    - Gathering
    - Scattering
  - Reductions
    - Counting
    - Comparisons
    - Extrema
    - Transformed Reductions
    - Logical
    - Predicates
  - Merging
  - Reordering
    - Partitioning
    - Stream Compaction
  - Prefix Sums
    - Segmented Prefix Sums
    - Transformed Prefix Sums
  - Set Operations
  - Sorting